

Technical Documentation

AI Career Companion Agent for Internship Matching and Interview Preparation -

System architecture, agent specifications, dataset and knowledge base

Submitted by

Afza Fathima

Computer Science and Engineering

Kalpataru Institute of Technology, Tiptur

AI Intern @Infosys Springboard

Contents

1. Purpose and Scope	3
2. System Architecture	4
2.1 Overview	4
2.2 Technology stack	4
2.3 Component responsibilities.....	5
3. Component Documentation.....	6
3.1 Frontend	6
3.2 Backend services (REST API)	6
3.3 Student profile management	7
3.4 Resume processing pipeline	7
3.5 Internship knowledge base, RAG architecture and vector index.....	8
3.6 LLM integration	8
3.7 Application tracking system	9
3.8 End-to-end data flow.....	9
4. AI Agent Specifications	11
4.1 Job-Resume Matching Agent.....	11
4.2 Skill Gap Analysis Agent.....	12
4.3 Resume and Cover Letter Customization Agent	12
4.4 Interview Preparation Agent	13
4.5 Conversational Career Assistant.....	13
5. Internship Dataset and Knowledge Base.....	14
5.1 Summary.....	14
5.2 Job categories.....	14
5.3 Data preprocessing.....	15
5.4 Chunking strategy.....	15
5.5 Embedding approach.....	15
5.6 Indexing and retrieval method	15
6. Configuration, Setup and Testing	16
6.1 Environment	16
6.2 Run the system	16
6.3 Run the tests and the demo	16
6.4 Known issues	16

1. Purpose and Scope

This document describes how the AI Career Companion works: its components, the data that moves between them, the responsibilities of each AI agent, and the internship knowledge base that grounds the system. It is written against the code in the repository (backend/app, frontend/src) and every figure quoted here was measured from the repository's own data files or reproduced by running its code.

AI Career Companion helps a student go from a resume to a tracked internship application: it extracts a structured profile from an uploaded resume, retrieves relevant internships from a knowledge base using RAG, scores the student against them, explains skill gaps, tailors resume and cover-letter content, prepares interview questions, answers follow-up questions in a chat assistant, and tracks applications through their lifecycle.

Design principle: deterministic where it must be exact, LLM where language is needed

Scores, gap classification, status validation, dashboard counts, reminders and intent validation are plain Python and fully unit-tested. The LLM is used only for resume parsing, written explanations and recommendations, document generation, interview questions and feedback, intent routing and chat synthesis, and each of those outputs is parsed and validated before it is used.

2. System Architecture

2.1 Overview

The system is a three-tier web application. A React single-page frontend calls a FastAPI backend over REST/JSON. The backend owns persistence (SQLAlchemy over SQLite), the RAG module (local vector index), the five AI agents, the deterministic application tracker, and the single outbound dependency, the Groq-hosted LLM.

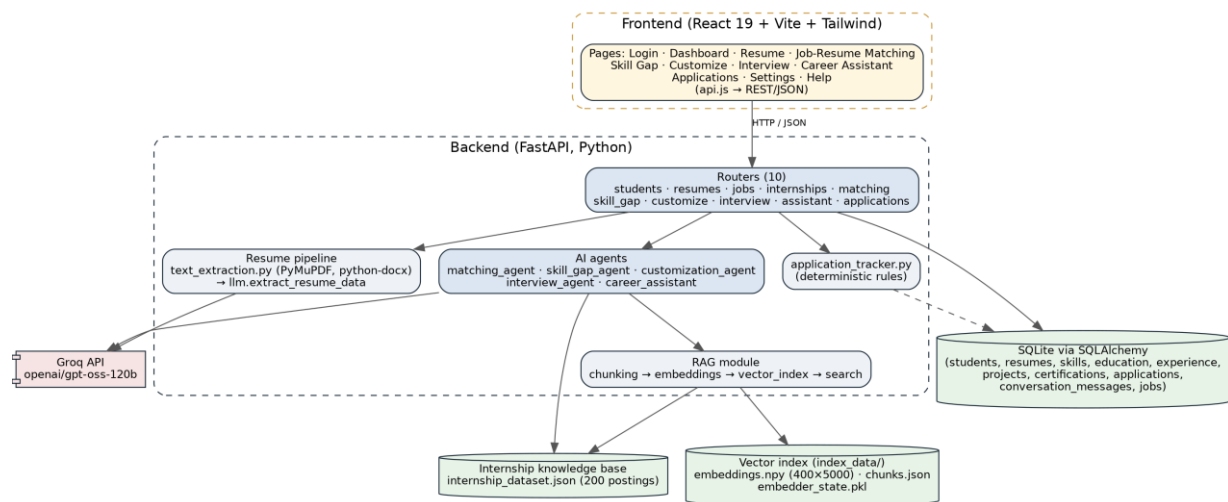


Figure 1. High-level system architecture

2.2 Technology stack

Layer	Technology	Notes
Frontend	React 19, Vite, Tailwind CSS 3, lucide-react	Page switching is state-based in App.jsx; react-router-dom is installed but not used for routing. API base URL is hard-coded to http://127.0.0.1:8000 in api.js.
Backend	FastAPI, Uvicorn, Pydantic v2	10 routers; CORS allows localhost/127.0.0.1 on ports 5173 and 5174.
Persistence	SQLAlchemy 2, SQLite (career.db)	DATABASE_URL env var switches to Postgres/Supabase with no code change. Tables are created with create_all (no migrations).
Resume text extraction	PyMuPDF (PDF), python-docx (DOCX)	Scanned/image-only PDFs yield no text and fail with a clear message.
LLM	Groq API, model openai/gpt-oss-120b	Model overridable via GROQ_MODEL; key via GROQ_API_KEY. JSON mode (response_format=json_object) on all structured calls.
Embeddings / index	scikit-learn TF-IDF (default); sentence-transformers all-MiniLM-L6-v2 (optional)	Selected by EMBEDDING_BACKEND. Index is a NumPy matrix plus JSON metadata, searched by brute-force cosine similarity.
Testing	pytest, FastAPI TestClient	backend/tests: unit tests for deterministic logic and an end-to-end workflow test with a fake LLM.

2.3 Component responsibilities

Component	Where	Responsibility
Frontend / interface	frontend/src	Login, Dashboard, Resume, Job-Resume Matching, Skill Gap, Customize, Interview, Career Assistant, Applications, Settings, Help pages; one function per endpoint in <code>api.js</code> .
Backend services	backend/app/routers	HTTP layer only: validate input, load the student profile as a plain dict, call an agent/module, shape the response.
Student profile management	models.py, routers/students.py	Student record plus skills, education, experience, projects and certifications; profile read as one aggregate.
Resume processing pipeline	routers/resumes.py, text_extraction.py, llm.py	Upload → store → extract text → LLM structured extraction → replace profile rows.
Internship knowledge base	data_pipeline/	Reproducible pipeline that turns the EMSCAD dataset into 200 structured internship postings.
RAG architecture	rag/	Chunking, embedding, indexing and semantic search over the knowledge base.
Vector database / index	rag/index_data/	<code>embeddings.npy</code> (400 × 5000), <code>chunks.json</code> , <code>embedder_state.pkl</code> .
AI agents	backend/app/*_agent.py, career_assistant.py	Matching, Skill Gap, Customization, Interview, Career Assistant (Section 4).
LLM integration	llm.py and each agent	One shared lazy Groq client; every agent builds its own prompt and validates the model's JSON.
Application tracking	application_tracker.py, routers/applications.py	Status vocabulary, dashboard metrics, reminders, filtering, CRUD; no LLM.

3. Component Documentation

3.1 Frontend

The student identity is stored in the browser's `localStorage` under `careerai_student`. Every page receives the student object and calls the backend through `api.js`.

Page	Backend endpoints used	What the student does
Login	POST /students (falls back to GET /students on 409)	Enters name, email and target role. Existing email returns the existing profile. The password field is not validated (no authentication yet).
Dashboard	none (static)	Placeholder: hard-coded greeting, stat cards (92% / 12 / 4) and four sample internships. Not yet connected to the API.
Resume	POST /resumes, POST /resumes/{id}/parse, GET /profile	Uploads PDF/DOCX, triggers parsing, views the extracted profile.
Job-Resume Matching	GET /recommended-internships	Sees ranked internships with score breakdown and explanation. The “Apply” button has no handler yet; applications are added from the Applications page.
Skill Gap	GET /skill-gap/{job_id}	Selects an internship and sees gap categories and recommendations.
Customize	POST /customize/resume cover-letter/{job_id}	Generates tailored content and regenerates with free-text feedback.
Interview	GET /interview-prep/{job_id}, POST /interview-prep/evaluate	Reads questions by category, submits an answer, receives a rating and feedback.
Career Assistant	POST /assistant/chat, GET /assistant/history	Chats; history persists across sessions.
Applications	CRUD, /dashboard, /reminders	Adds (company, role, status, deadline), filters, updates and deletes tracked applications; dashboard metrics and reminders are shown from the API. Resume/cover-letter snapshots are supported by the API but not yet saved from the UI.
Settings / Help	none	Settings is a static form with local state and hard-coded sample values (changes are not saved); its Logout button works. Help is static content.

3.2 Backend services (REST API)

Method and path	Purpose
POST /students · GET /students · GET /students/{id} · PATCH /students/{id}	Create (409 on duplicate email), list, read, update name/target role.
GET /students/{id}/profile	Student with skills, education, experience, projects, certifications.
POST /students/{id}/resumes	Upload resume. 415 if not PDF/DOCX; 413 above 5 MB. Stored under a UUID filename; parse_status = pending.
GET /students/{id}/resumes	List resumes, newest first.
POST /students/{id}/resumes/{rid}/parse	Extract text, call the LLM, replace the student's profile rows; 502 with the reason on failure.

Method and path	Purpose
GET /internships/search?q=&top_k=	Semantic search over the knowledge base.
GET /students/{id}/recommended-internships?top_k=&with_reasoning=	Matching agent output (retrieve, score, explain).
GET /students/{id}/skill-gap/{job_id}?with_recommendations=	Skill Gap agent output.
POST /students/{id}/customize/resume/{job_id} · .../cover-letter/{job_id}	Customization agent; optional feedback in the body.
GET /students/{id}/interview-prep/{job_id}?questions_per_category= · POST .../interview-prep/evaluate	Interview questions with revision topics; answer evaluation.
POST /students/{id}/assistant/chat · GET .../assistant/history	Career Assistant turn; stored conversation.
POST/GET /students/{id}/applications · GET/PATCH/DELETE .../{aid} · GET .../dashboard · GET .../reminders	Application tracking.
POST /jobs · GET /jobs · GET /students/{id}/matches	Legacy early-milestone keyword matcher over a small seeded jobs table. Superseded by the RAG-based matching agent; no page in the current UI uses it.

3.3 Student profile management

A Student owns five child collections, all cascade-deleted with the student. Profile data is created by resume parsing; the only fields editable through the API afterwards are name and target role.

Table	Key fields
students	id (UUID), name, email (unique), target_role, created_at, updated_at
resumes	student_id, filename, file_path, file_type (pdf/docx), parse_status (pending/success/failed), uploaded_at, parsed_at
skills	student_id, name, category (technical / soft / tool)
education	institution, degree, field, start_date, end_date (free-text dates)
experience	title, organization, description, start_date, end_date
projects	title, description, technologies (comma-separated string)
certifications	name, issuer (added in M3.1 for gap analysis)
applications	job_id, company, title, status, application_date, deadline, interview_date, interview_status, notes, resume_snapshot, cover_letter_snapshot
conversation_messages	student_id, role (user/assistant), content, intent, referenced_job_id, created_at
jobs (legacy)	title, company, location, description, required_skills (comma-separated)

Re-parsing a resume deletes and recreates the student's skills, education, experience, projects and certifications. There is no merge with earlier data and no manual profile editing yet.

3.4 Resume processing pipeline

1. **Upload:** content type must be PDF or DOCX, size ≤ 5 MB; file saved as `uploads/<uuid>.<ext>`; row created with status pending.

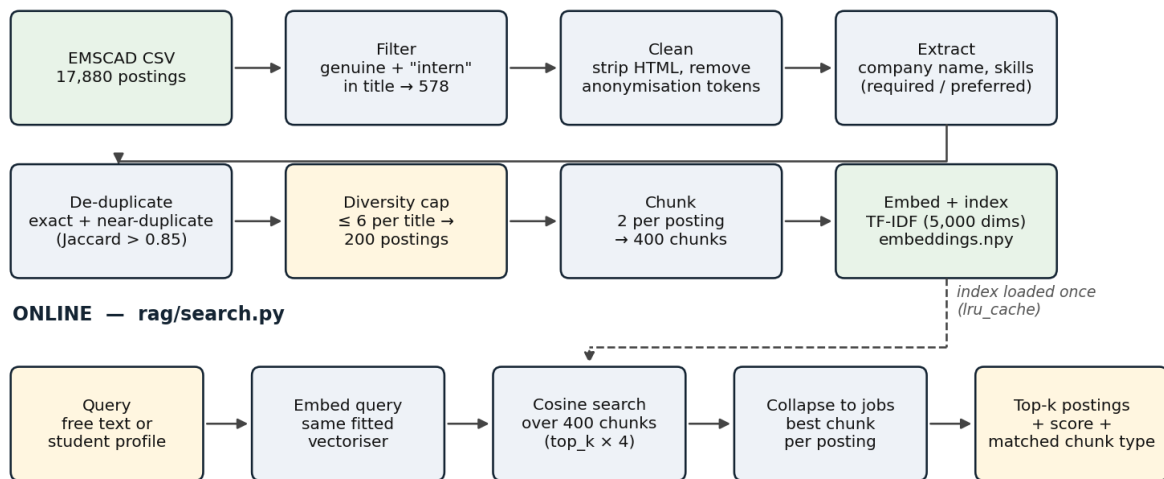
2. **Text extraction:** PyMuPDF concatenates page text for PDFs; python-docx joins paragraph text for DOCX. Empty text is rejected (likely a scanned image).
3. **Structured extraction:** the first 12,000 characters go to the LLM at temperature 0.1 in JSON mode with a prompt that forbids inventing skills, roles, projects, certifications or dates. Output keys: skills (technical/soft/tool), education, experience, projects (with technologies list), certifications.
4. **Persist:** old profile rows are deleted and new rows inserted in one transaction; status becomes success and `parsed_at` is set. Any exception sets status failed and returns 502 with the reason.

Not validated: the content type is taken from the upload header, not from file signature bytes; the prompt's "don't invent" rule is not post-checked for resume extraction (it is, for generated content, see 4.3).

3.5 Internship knowledge base, RAG architecture and vector index

Described in full in Section 5. In short: 200 internship postings are split into 400 chunks, embedded, and stored in a local NumPy index that is searched by cosine similarity.

OFFLINE — `build_dataset.py`, `build_index.py`



ONLINE — `rag/search.py`



Figure 2. Knowledge-base build (offline) and retrieval (online)

3.6 LLM integration

`llm.py` holds a lazily created Groq client shared by all agents. Each agent formats its own prompt, requests JSON mode where structured output is needed, and handles the response defensively.

Call	Module	Temp.	Output handling
Resume extraction	<code>llm.py</code>	0.1	JSON parsed; missing keys defaulted to empty lists; invalid JSON raises and marks the resume failed.
Match explanation	<code>matching_agent</code>	0.3	Falls back to a fixed "reasoning unavailable" string.
Gap recommendations	<code>skill_gap_agent</code>	0.3	Falls back to empty recommendations and a notice.
Tailored resume	<code>customization_agent</code>	0.4	On bad JSON returns an error object; otherwise adds a hallucination check.
Cover letter	<code>customization_agent</code>	0.5	Same as above.
Interview questions	<code>interview_agent</code>	0.5	Category-coverage check; on bad JSON returns empty questions with the revision topics.

Call	Module	Temp.	Output handling
Answer evaluation	interview_agent	0.3	Falls back to rating “unknown”.
Intent routing	career_assistant	0.1	validate_intent forces unknown intents to general_question and drops non-integer job IDs.
Chat synthesis	career_assistant	0.6	Plain text; instructed to use only the gathered data.

Prompt inputs are truncated to bound cost: job descriptions to 800 characters, resume text to 12,000, chat history to the last six messages in the prompt (ten loaded from the database).

3.7 Application tracking system

`application_tracker.py` contains no LLM and no database access, so its rules are testable in isolation; the router adapts ORM rows to dicts at the boundary.

- **Status stages (10):** Saved, Planning to apply, Applied, Application under review, Shortlisted, Interview scheduled, Interview completed, Offer received, Rejected, Withdrawn. Invalid values are rejected with 422 listing the valid options.
- **Terminal statuses:** Offer received, Rejected, Withdrawn (excluded from “active” and from deadline reminders).
- **Dashboard metrics (6):** total, active, upcoming deadlines (next 7 days), interviews scheduled, offers received, rejected.
- **Reminders (4):** upcoming deadlines; upcoming interviews (status Interview scheduled and date within the window); follow-ups suggested (status Applied for ≥ 10 days); pending applications (Saved or Planning to apply). Window is configurable (1–90 days).
- **Filtering:** by company and title (substring, case-insensitive), status, and deadline before/after.
- **Snapshots:** each application can store the resume and cover-letter text used, via `resume_snapshot` and `cover_letter_snapshot`.

3.8 End-to-end data flow

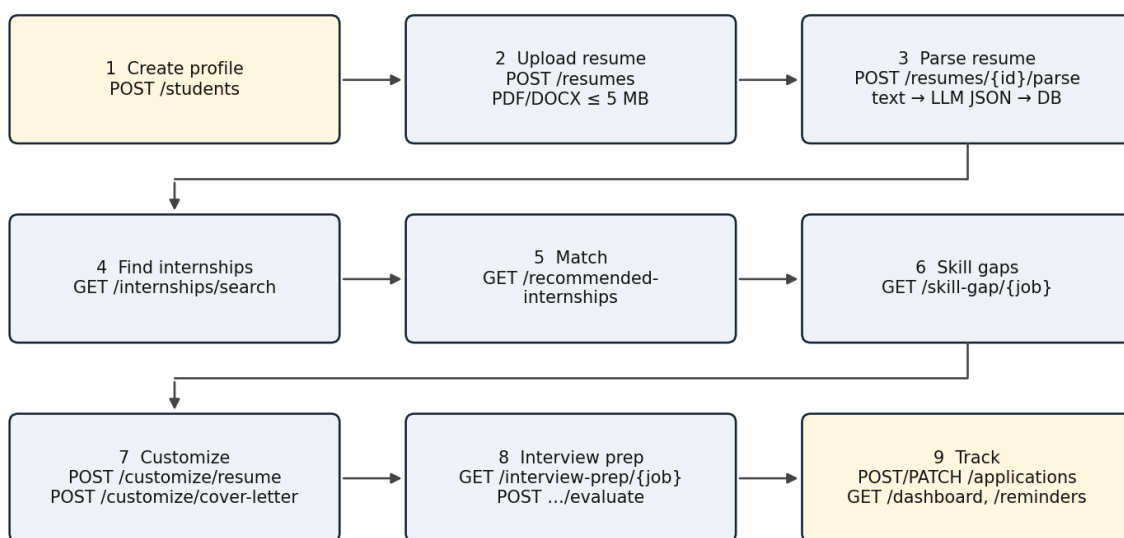


Figure 3. Student workflow and the endpoint behind each step

Step	Reads	Produces / writes
Create profile	request body	students row
Upload resume	PDF/DOCX file	file on disk; resumes row (pending)
Parse resume	file text	skills, education, experience, projects, certifications rows; resume status
Find internships	query text; vector index; dataset	ranked postings with similarity score
Match	student profile; vector index; dataset	top-5 postings with overall/skill/education/experience scores and an explanation
Skill gap	student profile; one posting	gap categories, experience/education gap, recommendations
Customize	profile; posting; ranked relevance	tailored resume JSON or cover letter plus validation flags
Interview prep	profile; posting; skill-gap result	questions in 5 categories, revision topics, answer feedback
Assistant	profile; last 10 messages; agent outputs	reply; two conversation_messages rows
Track	application fields; optional snapshots	applications rows; dashboard and reminders computed on read

4. AI Agent Specifications

The system is a multi-agent design in which specialised agents share one student profile and one knowledge base. The Career Assistant acts as the coordinator: it classifies the request, calls the relevant specialist agents, and writes the final reply from their real output. Specialists also reuse each other: the Interview agent builds its revision topics from the Skill Gap agent's result rather than re-deriving gaps.

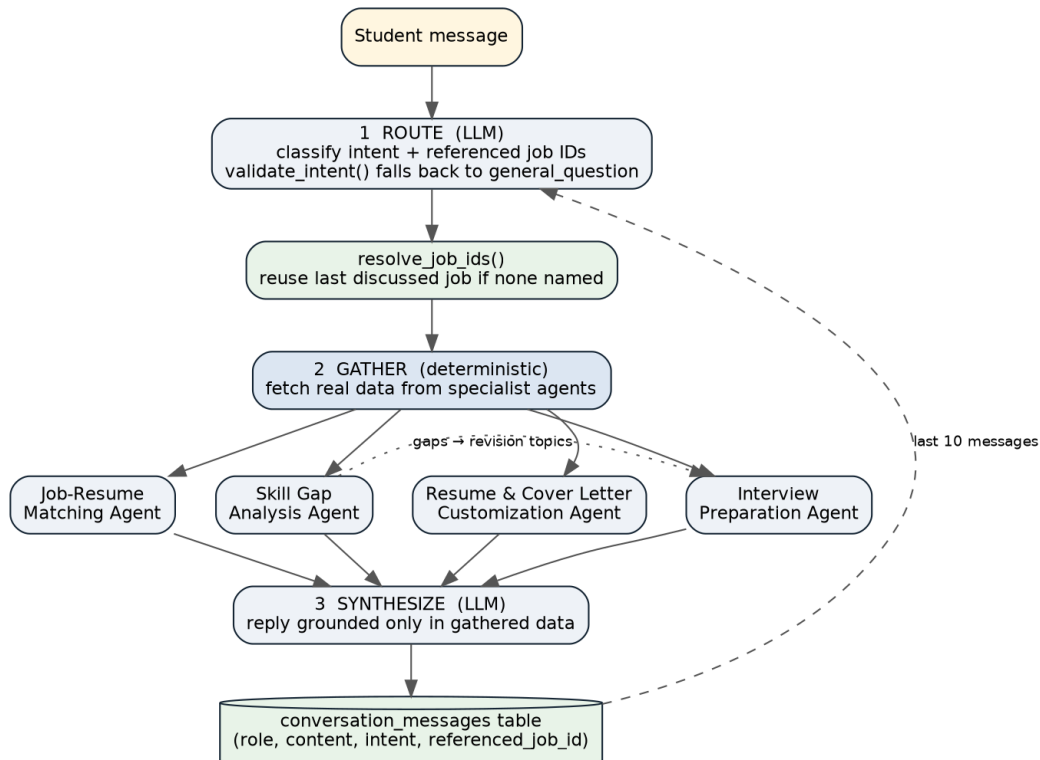


Figure 4. Conversational Career Assistant coordinating the specialist agents

4.1 Job-Resume Matching Agent

File: matching_agent.py · **Endpoint:** GET /students/{id}/recommended-internships

Aspect	Specification
Responsibility	Find internships that fit a student and say why, with an honest score breakdown.
Inputs	Student profile (target role, skills, education, experience, projects); parameters top_k (default 5, max 20) and with_reasoning (default true).
Stage 1: Retrieve	Builds a query from target role + skill names + experience titles + project titles, runs semantic search, takes the top 15 postings (broad recall).
Stage 2: Score	Skill score = $0.8 \times (\text{required skills matched} \div \text{required}) + 0.2 \times (\text{preferred matched} \div \text{preferred})$. Education score: 100 if no requirement or high-school level or met, 60 if a lower completed degree, 40 if no completed degree on record. Experience score: 100 for internship/entry-level/unstated, 80 for other, 70 for Associate, 20 for Mid-Senior/Director/Executive. Overall = $0.6 \times \text{skill} + 0.2 \times \text{education} + 0.2 \times \text{experience}$.
Stage 3: Explain	For the top results only, the LLM writes a 2–3 sentence explanation of the already-computed breakdown (told not to recompute it).
Outputs	Per internship: job_id, title, company, location, overall_score, skill_score, education_score, experience_score, matched_required_skills, missing_required_skills, matched_preferred_skills, retrieval_score, reasoning.

Aspect	Specification
Guardrails	Scoring is deterministic and does not depend on the LLM; reasoning failure degrades to a fixed message. Skill comparison is case-insensitive exact match.
Known limits	Exact-string skill matching (no synonyms such as JS vs JavaScript); crude degree matching; see Section 13 of the report.

4.2 Skill Gap Analysis Agent

File: skill_gap_agent.py · **Endpoint:** GET /students/{id}/skill-gap/{job_id}

Aspect	Specification
Responsibility	For one chosen internship, classify what the student is missing and recommend how to close each gap.
Inputs	Student profile (including certifications); one posting from the knowledge base; with_recommendations flag.
Deterministic classification	critical_missing: required skill with no evidence and no related skill. partially_demonstrated: required skill absent but a related skill is held (e.g. has JavaScript, job wants TypeScript), using a deliberately small 16-entry SKILL_FAMILIES table. matched_required, matched_preferred, preferred_gaps. Also experience_gap (none/minor/moderate/major) and education_gap (including “in progress” for current students).
LLM step	For every critical or partial gap: why it matters for this role and a concrete, non-product-specific recommendation; plus a 2–3 sentence honest overall summary.
Outputs	The classification fields above, skill_recommendations [{skill, why_it_matters, recommendation}], overall_summary.
Guardrails	Family table is conservative on purpose (a false “related skill” is worse than none). Recommendations degrade to an empty list on invalid JSON.

4.3 Resume and Cover Letter Customization Agent

File: customization_agent.py · **Endpoints:** POST /students/{id}/customize/resume/{job_id} and .../cover-letter/{job_id}

Aspect	Specification
Responsibility	Tailor the student’s existing content to one posting without inventing anything.
Inputs	Profile (skills, certifications, education, experience with descriptions, projects with technologies); posting (required/preferred skills, first 800 characters of description); optional feedback text for regeneration.
Rank (deterministic)	rank_relevant_content scores skills, experience and projects by keyword overlap with the posting so the LLM works from a pre-prioritised list.
Generate (LLM)	Resume JSON: headline, prioritized_skills, experience_bullets, project_highlights, keywords_incorporated. Cover letter: 3–4 paragraphs referencing real skills and projects by name. Prompts state that the profile is the only source of truth.
Validate (deterministic)	validate_no_hallucination flags skill-like terms from a closed vocabulary that appear in the output but nowhere in the student’s profile, returning {clean, suspect_skills}.
Outputs	Generated content plus _validation.
Limits	The check reliably catches invented skills from its ~26-term vocabulary. It cannot catch invented achievements or exaggeration written in prose (for example “led a team of 10”). Stated in the code and repeated here.

4.4 Interview Preparation Agent

File: interview_agent.py · **Endpoints:** GET /students/{id}/interview-prep/{job_id}, POST .../interview-prep/evaluate

Aspect	Specification
Responsibility	Generate role-specific interview questions with preparation guidance, flag topics to revise, and give feedback on practice answers.
Inputs	Profile; posting; the Skill Gap agent's result; questions_per_category (1–5, default 2). For evaluation: question and answer text.
Revision topics (deterministic)	From the gap result: critical missing skills → high priority; partially demonstrated → medium; an experience gap adds "Experience level expectations" (high if major).
Questions (LLM)	Exactly N per category across five categories: Technical, Resume-based, Project-based, Role-specific, HR/general, each with 1–2 sentences of guidance. Revision topics are given to the model so questions probe honest handling of gaps.
Validation	validate_question_coverage verifies all five categories are present and reports counts per category.
Answer evaluation (LLM)	Returns rating (strong / adequate / needs_work), feedback, strengths, improvements; the student's real background is included so inconsistent answers can be noticed.
Outputs	questions, revision_topics, _coverage; or rating/feedback/strengths/improvements.

4.5 Conversational Career Assistant

File: career_assistant.py · **Endpoint:** POST /students/{id}/assistant/chat

Aspect	Specification
Responsibility	Let the student ask for any of the above in natural language, with replies grounded in real agent output and remembered across turns.
Inputs	Message; profile; last 10 stored messages; job IDs discussed earlier (from referenced_job_id).
1. Route (LLM)	Classifies one of seven intents: recommend_internships, explain_match, explain_skill_gap, compare_internships, customize_materials, interview_prep_summary, general_question, and extracts referenced job IDs. validate_intent normalises the result.
Context retention	resolve_job_ids: if the message names no job but one was discussed earlier, the last discussed job becomes the subject. Messages are persisted with intent and job ID.
2. Gather (deterministic dispatch)	Calls the existing agents: matching (recommend), score_candidate (match/compare), analyze_skill_gap (gap), customization (resume or cover letter, chosen by the words "cover letter"), interview agent (one question per category). Unknown or job-less cases fall back to semantic search.
3. Synthesize (LLM)	Writes a conversational 2–5 sentence reply using only the gathered data; asks a clarifying question if a job is needed but none was identified.
Outputs	{reply, intent, referenced_job_ids}; two rows appended to conversation_messages.

5. Internship Dataset and Knowledge Base

5.1 Summary

Item	Value (measured from the repository files)
Sample job postings	200 in <code>internship_dataset.json</code> (176 distinct titles)
Source	EMSCAD (Employment Scam Aegean Dataset): 17,880 real job postings from 2012–2014, University of the Aegean, obtained from a GitHub CSV mirror of the Kaggle “Real / Fake Job Posting Prediction” data
Collection method	Filtered from the public dataset by <code>build_dataset.py</code> ; no scraping and no live job feeds
Knowledge-base chunks	400 (200 overview + 200 requirements)
Embedding	TF-IDF, English stop-words removed, 5,000 features, unit-normalised (default). Optional sentence-transformers all-MiniLM-L6-v2
Index	Dense matrix $400 \times 5,000$ (float64, ≈ 16 MB) in <code>embeddings.npy</code> ; metadata in <code>chunks.json</code> (≈ 395 KB); fitted vectoriser in <code>embedder_state.pkl</code> . Brute-force cosine similarity
Skill vocabulary	60-term list (<code>skill_vocabulary.py</code>); 58 distinct skills occur in the 200 postings

5.2 Job categories

The EMSCAD “function” field is missing for 70 of 200 postings, so categories below describe the 130 postings that carry one; the remaining 70 are classified by title and skills only.

Function (as labelled in source)	Postings	Function (continued)	Postings
Marketing	34	Writing/Editing	5
Engineering	17	Consulting	5
Design	11	Business Development	5
Information Technology	10	Customer Service	4
Sales	9	Other labelled functions	17
Art/Creative	7	Unlabelled	70
Data Analyst	6		

Most common industries: Marketing and Advertising (24), Internet (22), Computer Software (16), Information Technology and Services (14), Design (9); 72 postings have no industry. Employment type: Full-time 63, Part-time 28, Temporary 27, Other 20, Contract 9, unstated 53. Countries: US 99, GB 29, GR 18, DE 12, NZ 8.

Stated requirements: experience level is “Internship” for 116 postings, Associate 8, Entry level 7, Mid-Senior 4, Director 2, unstated 63. Education: Bachelor’s 43, Some college 25, Master’s 8, High school 4, Associate 2, other 2, unspecified or missing 116. Most frequent required or preferred skills: Communication (117), Sales (51), Excel (51), JavaScript (36), Word (34), Photoshop (33), CSS (31), HTML (31).

5.3 Data preprocessing

#	Step	Detail
1	Filter	Keep genuine postings (<code>fraudulent == 0</code>) whose title contains “intern”: 578 of 17,880.
2	Clean	Strip HTML tags, unescape entities, remove the dataset’s #URL_...# and #EMAIL_...# anonymisation tokens, collapse whitespace.
3	Company name	Regex patterns over the company-profile text (“We’re X”, “X is a ...”, “At X, ...”). Falls back to “Undisclosed Company” (104 of 200).
4	Skill extraction	Keyword match against the vocabulary; a sentence containing a “preferred / nice-to-have / plus” marker moves its skills from required to preferred. “Go” and standalone “R” were excluded after testing showed they matched ordinary words.
5	De-duplicate	Exact title + description duplicates dropped (33). A later fix added near-duplicate detection within each company: word-set Jaccard similarity above 0.85 drops the shorter posting (63 removed, per the data-pipeline README).
6	Diversity cap	At most 6 postings per exact title so repeated titles do not dominate.
7	Trim	Sort by skill richness and keep 200.

Record schema: `source_job_id`, `title`, `company`, `location`, `description`, `requirements_raw`, `required_skills[]`, `preferred_skills[]`, `experience_requirement`, `education_requirement`, `industry`, `function`, `employment_type`.

5.4 Chunking strategy

Field-based rather than fixed-size: each posting yields an **overview** chunk (title, company, description) describing what the role is, and a **requirements** chunk (requirements text, required and preferred skills, experience level, education) describing what it needs. Each chunk is capped at 1,500 characters. The median description is about 1,260 characters, so most postings fit without truncation. This keeps chunks semantically coherent, so a question about skills and a question about the company retrieve different chunks. 200 postings → 400 chunks.

5.5 Embedding approach

Two interchangeable backends share one interface (fit, encode → unit-normalised array, so cosine similarity is a dot product), selected by `EMBEDDING_BACKEND`:

- **tfidf** (default, and the one built and evaluated): offline, no downloads, matches on term overlap. Weakness: no word-sense or paraphrase awareness (for example “react” the verb versus React the framework).
- **sentence-transformers** (`all-MiniLM-L6-v2`): the semantic option specified in the design. The code path exists but has **not been run or evaluated** in this project’s test environments; enable it, rebuild the index and re-run the evaluation before relying on it.

5.6 Indexing and retrieval method

`build_index` chunks the dataset, fits the embedder on all chunk texts, encodes them and saves the matrix, metadata and embedder state. At query time the index is loaded once (`lru_cache`), the query is embedded with the same vectoriser, cosine similarity is computed against all 400 chunks, the top (`top_k × 4`) chunks are kept, and results are collapsed to one entry per posting using its best-scoring chunk. Brute force is appropriate at this size; a corpus of tens of thousands of chunks would need an approximate index (FAISS) or pgvector.

6. Configuration, Setup and Testing

6.1 Environment

Variable	Default	Purpose
GROQ_API_KEY	(required for LLM features)	Resume parsing, explanations, customization, interview, assistant.
GROQ_MODEL	openai/gpt-oss-120b	LLM used by every agent.
DATABASE_URL	sqlite:///./career.db	Switch to Postgres/Supabase.
EMBEDDING_BACKEND	tfidf	tfidf or sentence-transformers.
EMBEDDING_MODEL	all-MiniLM-L6-v2	Model for the sentence-transformers backend.

6.2 Run the system

```
# backend (from backend/)
python -m venv venv && venv\Scripts\activate          # Windows
pip install -r requirements.txt scikit-learn numpy
copy .env.example .env          # then set GROQ_API_KEY
python -m app.rag.build_index          # only if the index is missing
uvicorn app.main:app --reload          # http://127.0.0.1:8000/docs

# frontend (from frontend/)
npm install && npm run dev          # http://localhost:5173
```

Note: `requirements.txt` does not list scikit-learn or numpy, which the RAG module needs; install them as shown (and sentence-transformers only if you switch backends).

6.3 Run the tests and the demo

```
pip install pytest httpx
python -m pytest tests -v          # 19 pass, 1 expected failure (documented
limitation)
python -m app.evaluation.run_eval          # retrieval precision@5 over 5 profiles
python demo/demo_workflow.py --resume path/to/resume.pdf # full workflow against the live
API
```

The end-to-end test drives the real app, database and RAG index but substitutes a deterministic fake for the Groq client, so it checks wiring, validation and persistence, not the quality of LLM text.

6.4 Known issues

Area	Issue	Effect
Frontend placeholders	Dashboard and Settings are static; Matching “Apply” button is inert; customised documents are not saved to an application from the UI.	Demonstrate those features through the other pages; the backend for all of them already exists.
Authentication	No real authentication; login by email only, password not checked.	Anyone who knows an email can open that profile. Acceptable for a local demo only.
Degree matching	Education scoring keys on the literal “bachelor’s”.	“Bachelor of Engineering”, “B.E.”, “B.Tech” score 40 instead of 100 on postings that require a Bachelor’s degree (tracked by an expected-failure test).

Area	Issue	Effect
Zero required skills	23 postings list no required skills; the skill score then starts at 80.	These can show 100/100 for a student with no matching skill (for example PINT “Web Design Intern” when its skills are all preferred).
Skill matching	Exact, case-insensitive string match.	Synonyms (JS/JavaScript, ReactJS/React) are missed.
Bullet-list parsing	Unpunctuated bullet text is treated as one sentence.	Some postings classify required skills as preferred.
Company names	104 of 200 are “Undisclosed Company”; at least one extraction artifact (“What”).	Display only; matching is unaffected.
Uploads in repository	backend/app/uploads holds 39 resume files and is not git-ignored.	Personal data should not be committed; remove and ignore the folder before publishing.
Dependencies	requirements.txt omits scikit-learn and numpy; .gitignore also excludes .env.example.	Fresh installs fail at the RAG import until installed.